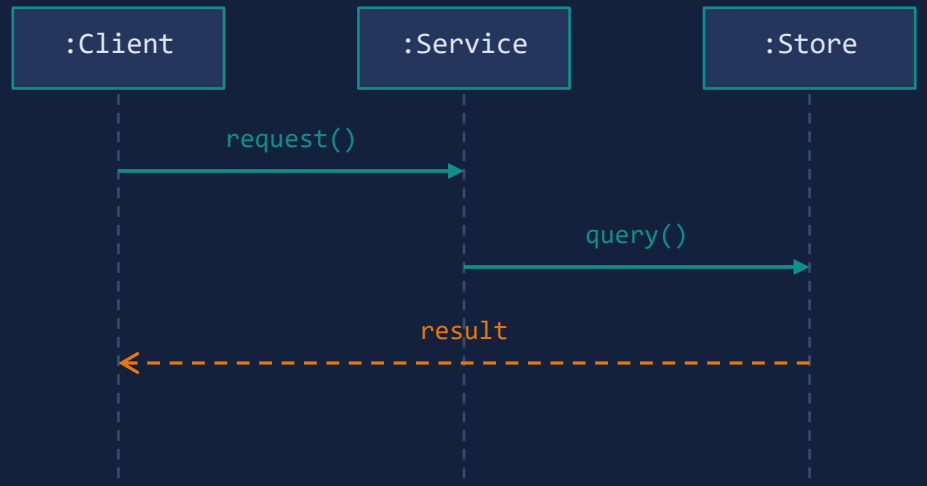
OBJECT-ORIENTED ANALYSIS & DESIGN

Sequence Diagrams

How objects talk to each other, and how to draw the conversation



The main question

You order food on a delivery app.

What happens behind the scenes after you tap “Place order”?

- Which systems / components exchange information?
- Who has to WAIT for an answer before doing anything else?
- What happens if one of them never answers?

While you talk, sketch it:

boxes = systems
arrows = “talks to”
X = waits for answer

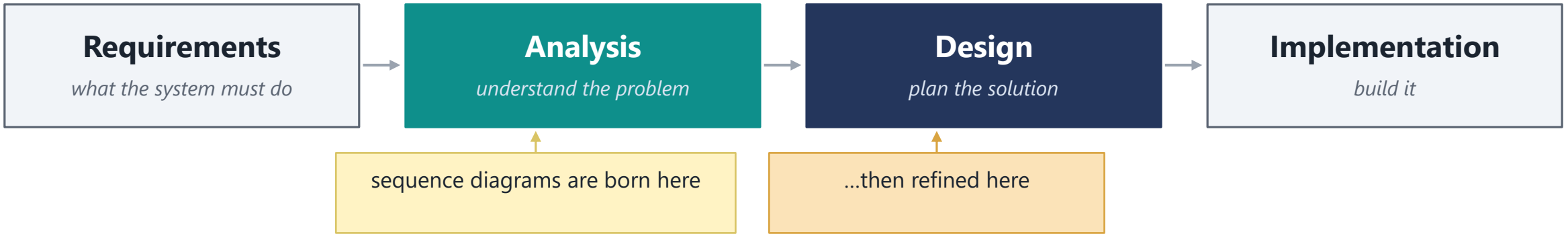
No UML needed yet,
plain boxes and arrows.

SECTION 1

Where sequence diagrams live

SDLC position · static vs dynamic · use cases → diagrams

Born in Analysis, raised in Design



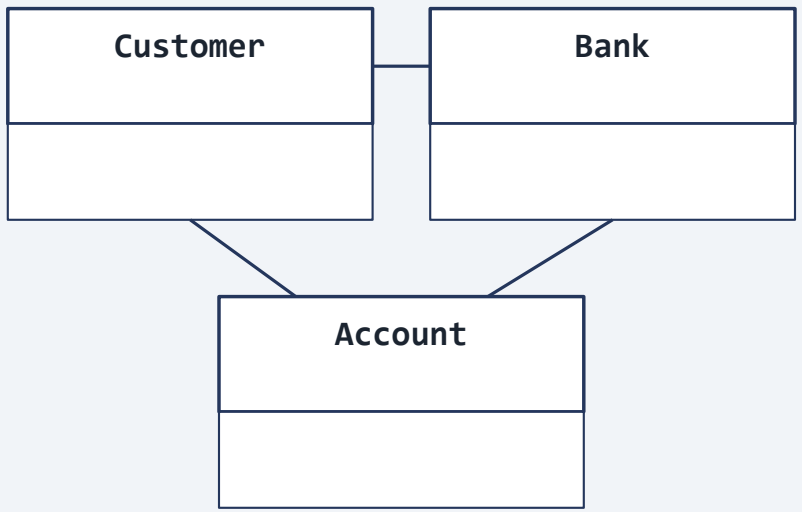
- **Analysis phase:** the problem domain is refined from requirements; the **behavioral model**, how the system behaves, is built here.
- A sequence diagram is a core artifact of the behavioral model: it captures **inter-object behavior ordered by time**.

- Design phase: same diagram, more concrete: real classes, boundaries, services.

Same system, two questions

Class diagram · STRUCTURE

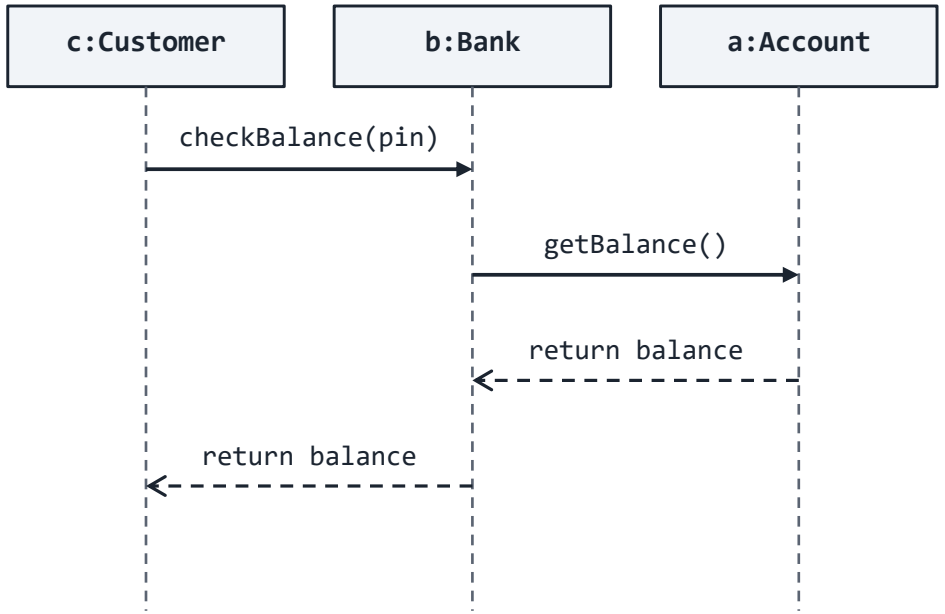
"What exists? What are the nouns?"



No time, no order, no behavior.

Sequence diagram · BEHAVIOR

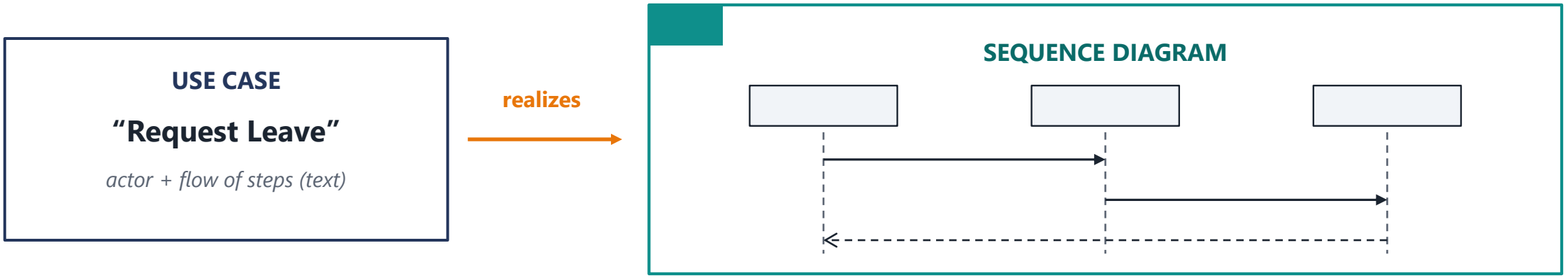
"What happens, in what order? The verbs."



Time flows ↓. Same three classes, new story.

One use case → one conversation script

the bridge from what you already know to today's topic

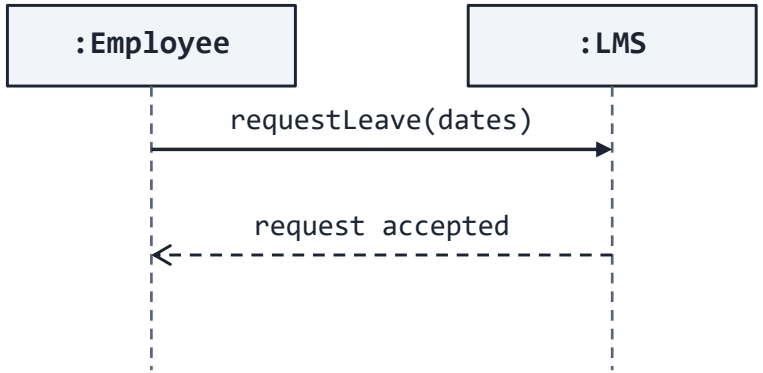


- A use case states a GOAL and a FLOW: main flow (happy path) + alternative flows.
- In UML, interaction diagrams (sequence diagrams) **realize** a use case, they show the objects that carry out its flow. In process terms, this is called a **use-case realization**.
- **Rule of thumb:** one scenario (one flow through one use case) ≈ one sequence diagram.

First pass black-box, second pass white-box

Step 1: System sequence diagram (analysis):

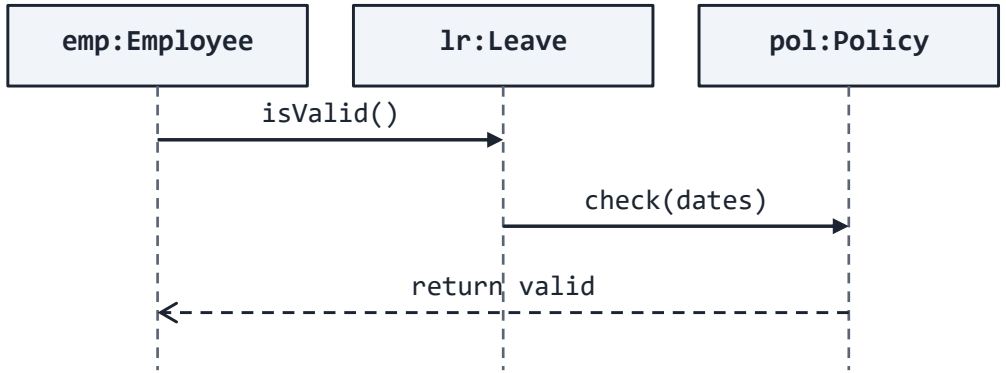
actor ↔ whole system as ONE lifeline; messages = use-case steps



open the box

Step 2: Design sequence diagram:

the system lifeline becomes the objects inside it



Why two passes?

The black-box pass keeps you honest about WHAT the system promises (straight from the use case).
The white-box pass decides WHO inside the system keeps each promise, that is design.

Running example: Leave Management System

our thread for the whole lecture: classes: Employee, Leave

USE CASE UC-01 · Request Leave

Actor: Employee

Main flow:

1. Employee enters dates and requests leave.
2. System creates a leave request.
3. System validates the request against leave policy.
4. System submits the request for approval.

Alternative flow: 3a. If the request is invalid, the system discards it and reports the reason.

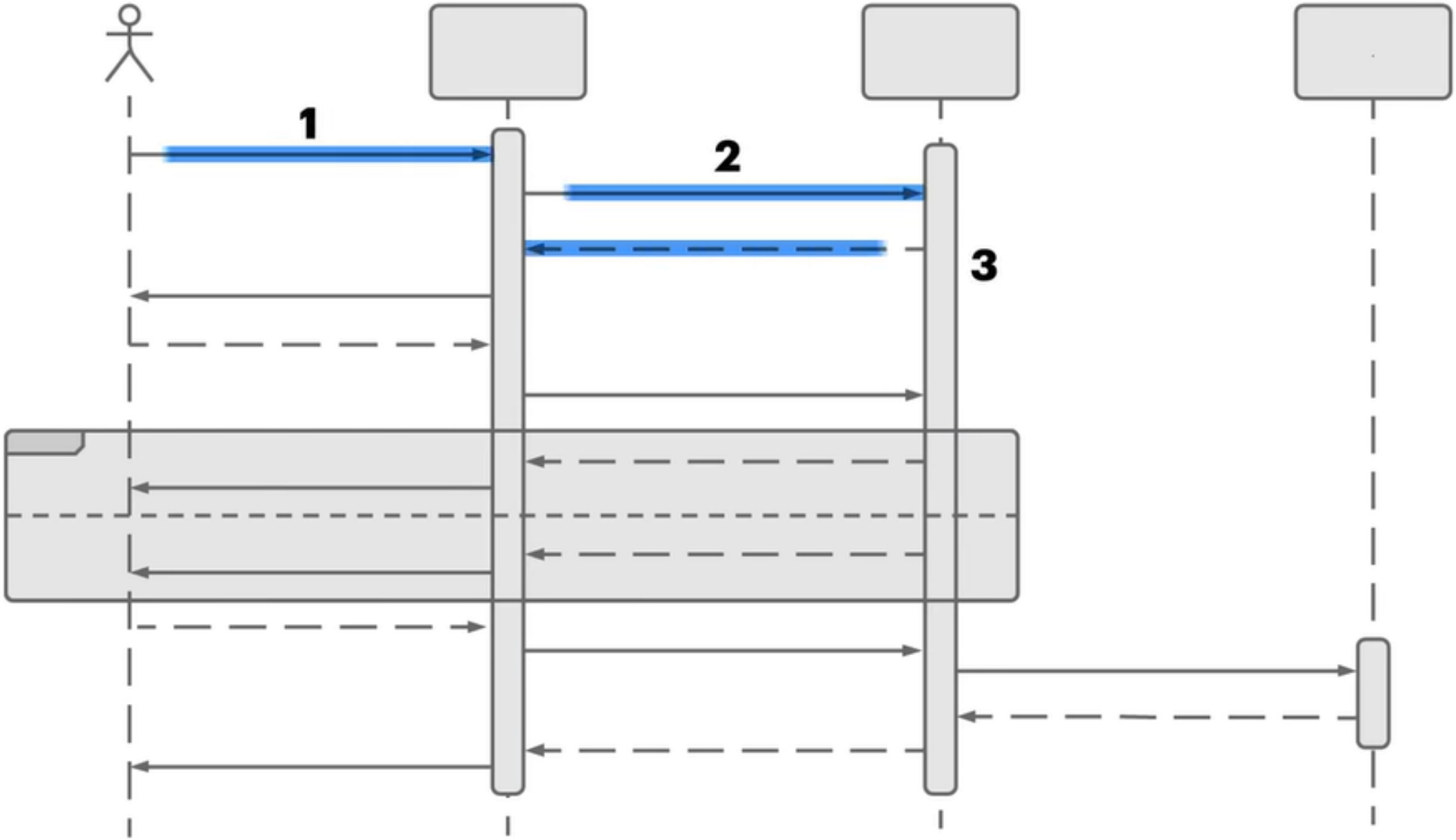
Watch for these steps

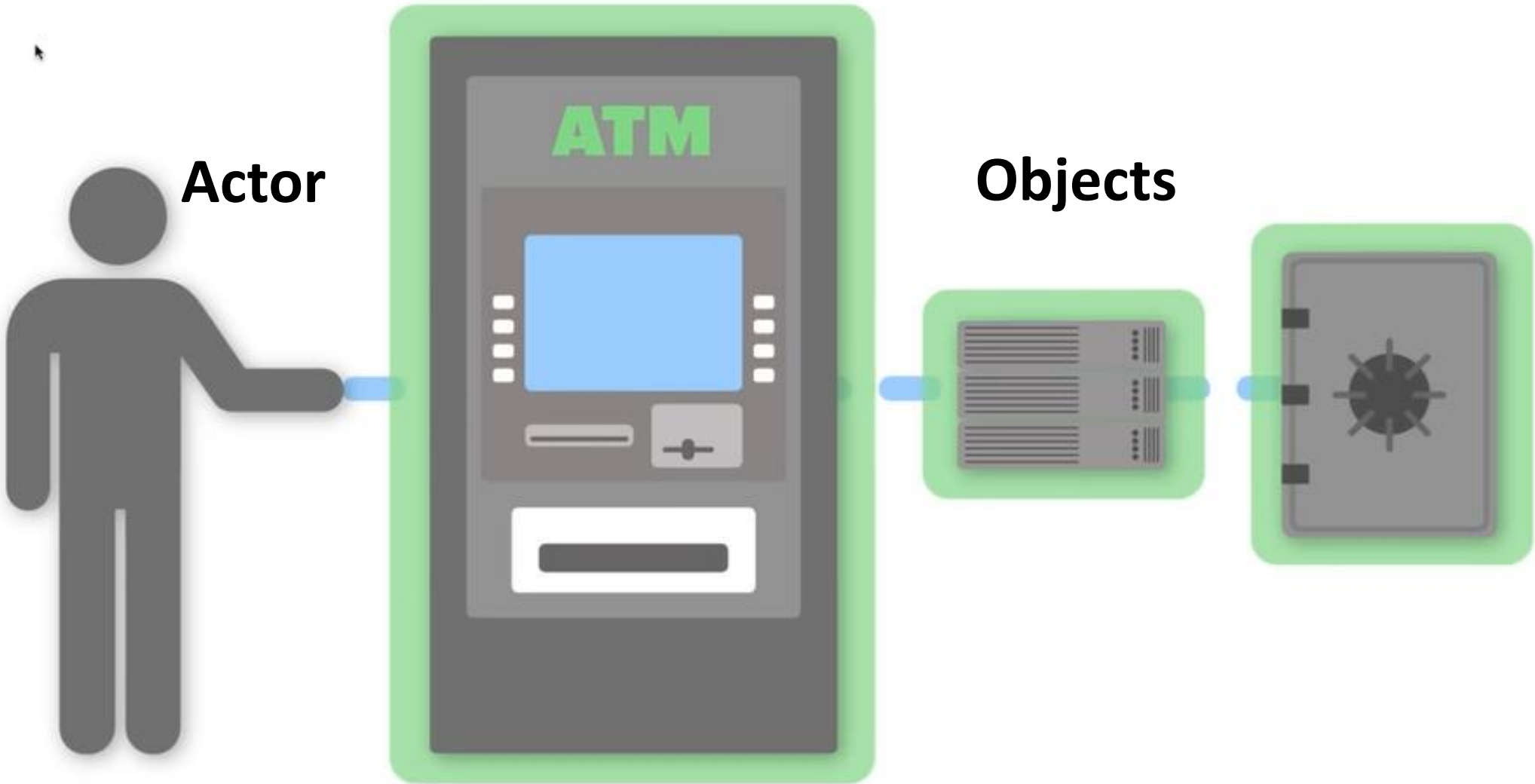
…they reappear as numbered messages on the diagram we build in Section 3:
② create message
③ sync call + reply
④ / 3a alt fragment

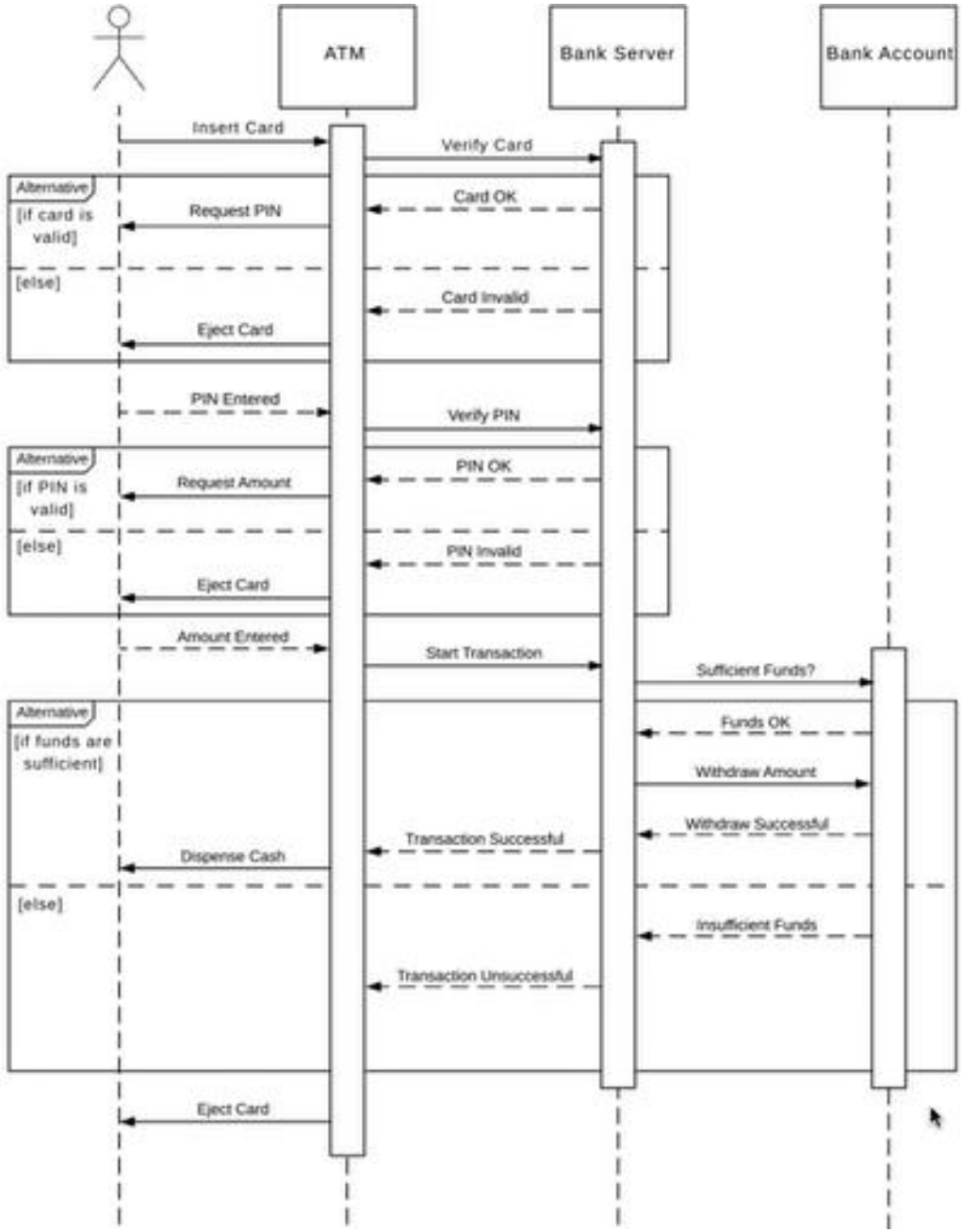
SECTION 2

Anatomy of an interaction

lifelines · messages by action type · lost & found


1
2
3

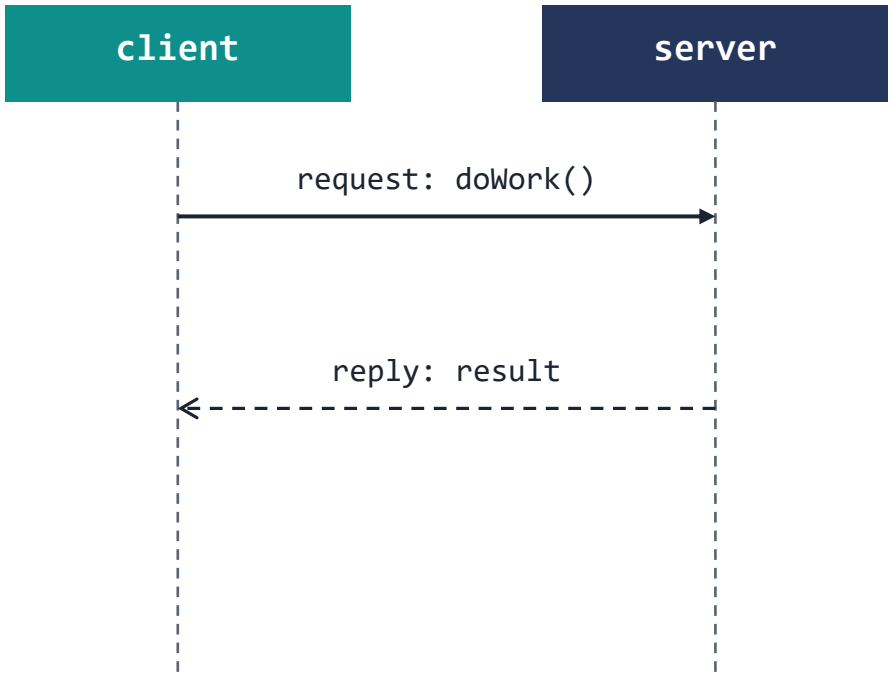

Actor
ATM
Objects


ATM
Bank Server
Bank Account
Insert Card
Verify Card
Alternative
[if card is valid]
Request PIN
Card OK
[else]
Card Invalid
Eject Card
PIN Entered
Verify PIN
Alternative
[if PIN is valid]
Request Amount
PIN OK
[else]
PIN Invalid
Eject Card
Amount Entered
Start Transaction
Sufficient Funds?
Alternative
[if funds are sufficient]
Funds OK
Withdraw Amount
Withdraw Successful
Transaction Successful
Dispense Cash
[else]
Insufficient Funds
Transaction Unsuccessful
Eject Card

No object is an island

the client–server view of computing

- Objects are acted upon, and act upon other objects.
- **Server role:** an object provides services (operations others can invoke).
- **Client role:** an object requests a service by **sending a message** (invoking an operation).
- A **contract** between client and server (the operation's signature & semantics) keeps the interaction correct.
- *Roles swap constantly: today's server is tomorrow's client.*



browser ↔ web server, app ↔ API, object ↔ object — same pattern

What a sequence diagram is

A UML behavior (interaction) diagram that shows how objects interact in one Scenario, the messages exchanged between lifelines, ordered by time (top → bottom).

Three ingredients:

1 LIFELINES

the participants
(who is in the conversation)

2 MESSAGES

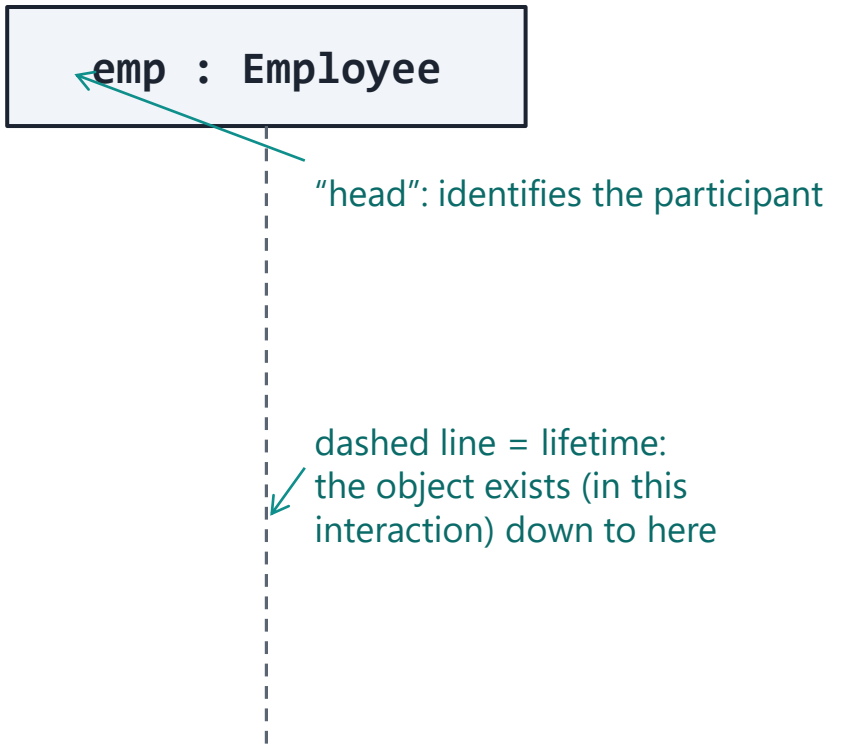
the communication
(what is sent, who waits)

3 FRAGMENTS

the logic
(branches, loops, options)

Ingredient 1 · Lifelines

one participant = one head + one lifetime



`ObjectName[selector] : ClassName`

all three parts optional except the colon rule below

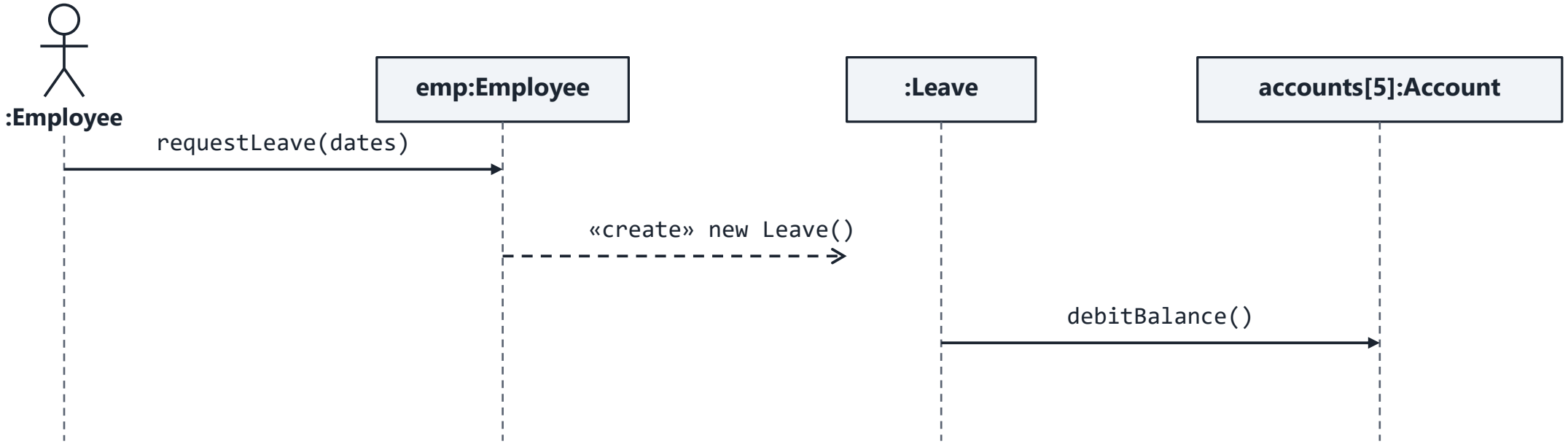
`emp : Employee`
named object: this specific instance

`: ShoppingCart`
anonymous: some instance of the class

`accounts[5] : Account`
selector: the 5th element of a multi-valued host

A lifeline always represents ONE interacting entity, even if the class has many instances.

Lifelines on a real diagram



actor participant (a person or external system in the talk)

named object

anonymous object

selector picks one instance of many

Drill

write the lifeline notation paper or chat

1. Your phone, as a named object of class Smartphone (call it myPhone).

2. Some email server, you don't care which instance.

3. The third element of an array of Student objects named students.

Syntax reminder: ObjectName[selector] : ClassName

Messages

A message = one specific kind of communication between lifelines:
an operation call, a signal, a creation or a destruction.

Two ways to classify them (UML):

BY ACTION TYPE

what the message DOES

- synchronous call
- reply / return
- delete
- asynchronous call / signal
- create

BY PRESENCE OF EVENTS

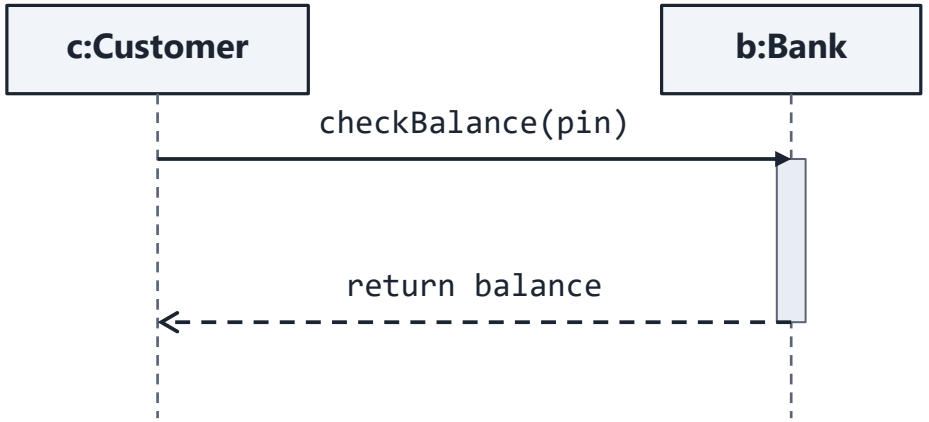
whether send & receive events exist

- complete
- found
- lost
- unknown (default)

We use action types every day; event presence matters when messages can vanish.

Synchronous call: “I wait for you”

solid line, FILLED arrowhead · caller suspends until reply



Customer does NOTHING else until the reply arrives

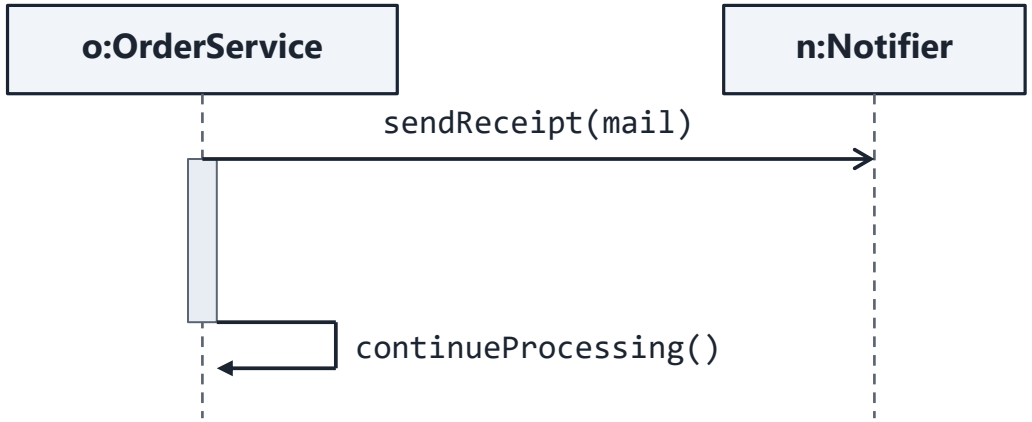
BankingApp.java

```java
// the caller BLOCKS on this line
double balance = bank.checkBalance(pin);
System.out.println("Balance: " + balance);
```

- Real situations: ATM verifying a PIN; an HTTP request/response; a function call in any language.
- Design smell check: every sync arrow is a place your system can **block**.

Asynchronous call: “fire and forget”

solid line, OPEN (half) arrowhead · sender continues immediately



OrderService keeps working
no reply arrow exists

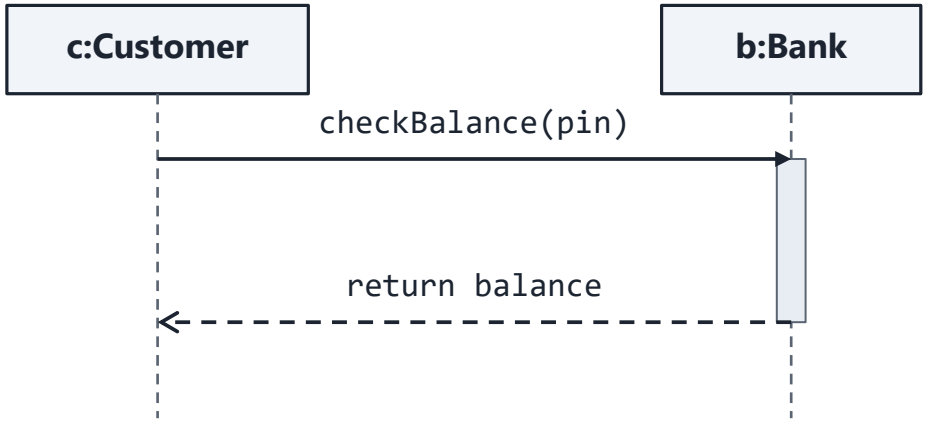
OrderService.java

```java
// publish and move on, no waiting
events.publish(new OrderPlacedEvent(orderId));
logger.info("order accepted");
```

- Real situations: e-mail/SMS sending, printing a receipt, publishing to a message broker (Kafka, RabbitMQ), UI event handlers.
- The open head says: **“no reply is expected on this arrow”**.

Reply / return: giving control back

dashed line, open arrowhead



Bank.java

```java
double checkBalance(String pin) {
    verify(pin);
    return balance;   // ← the reply arrow
}
```

- Carries control (and usually data) back to the caller.
- Often LEFT OUT of diagrams: for a sync call the reply is implied. Draw it when the returned value matters to the story.

Create: an object enters the story

dashed line, open arrowhead, aimed at the new HEAD



lr's lifetime STARTS where the arrow lands, not at the top

Employee.java

```
LeaveRequest lr = new LeaveRequest(emp, dates);
```

- The `new` operator (Java/C++), a constructor call, is a create message.
- Some tools write the stereotype **«create»** on a plain message instead of a dashed arrow, same meaning.
- Checkout → a new Order object; Add-to-cart → a new CartLine object.

Delete: an object exits the story

solid line, filled arrowhead; lifeline ends in an X



the X ends the lifetime: no messages below it

two languages

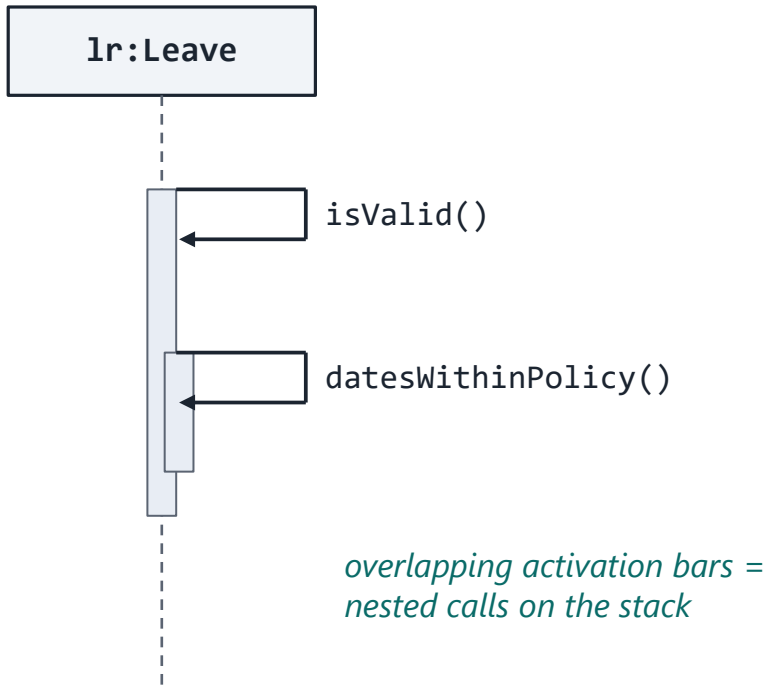
```cpp
// C++: explicit destruction
delete session;

// Java/Python: no delete keyword —
// model end of LIFE, e.g.
session.invalidate();  // GC reclaims later
```

- In managed languages the X models **logical death** (session invalidated, file closed), not memory free.

Self message: talking to yourself

a method invoking another method on the SAME object



Leave.java

```java
boolean isValid() {
    return datesWithinPolicy();
}
```

- Drawn as a loop-back arrow beside the lifeline.
- Use sparingly: only when the internal call matters to the story (recursion, helper steps, callbacks).

The message palace: one glance

arrow	name	notation	in code	who waits?
	synchronous call	solid + filled head	`x = f(y)`	**caller waits**
	**asynchronous call**	solid + open head	`events.publish(e)`	nobody waits
	reply / return	dashed + open head	`return v`	—
	**create**	dashed + open head → head	`new T(...)`	—
	**delete**	solid + filled head, X ends life	`delete p`	—

Plus the self-message (loop-back arrow) for object-internal calls.

When messages vanish: lost & found

classification by presence of send / receive events

complete

send event + receive event both known

the normal case

lost

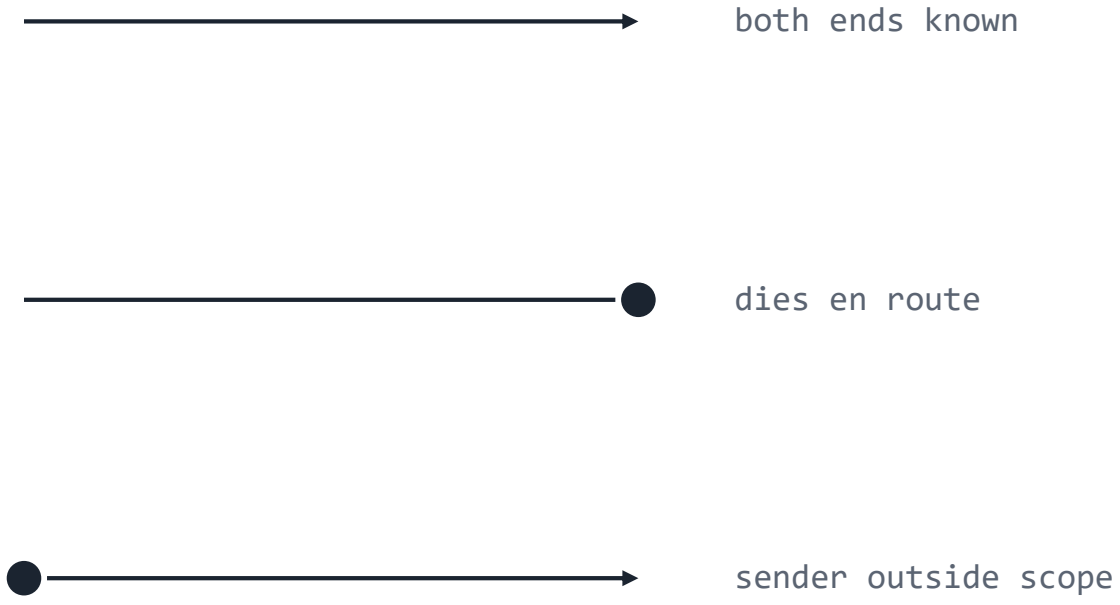
send event known, receive event MISSING

message may never arrive

found

receive event known, send event UNKNOWN

sender outside diagram scope



Real world: a UDP datagram that never arrives is a LOST message; a timer/cron trigger appearing “from nowhere” inside your scope is a FOUND message.

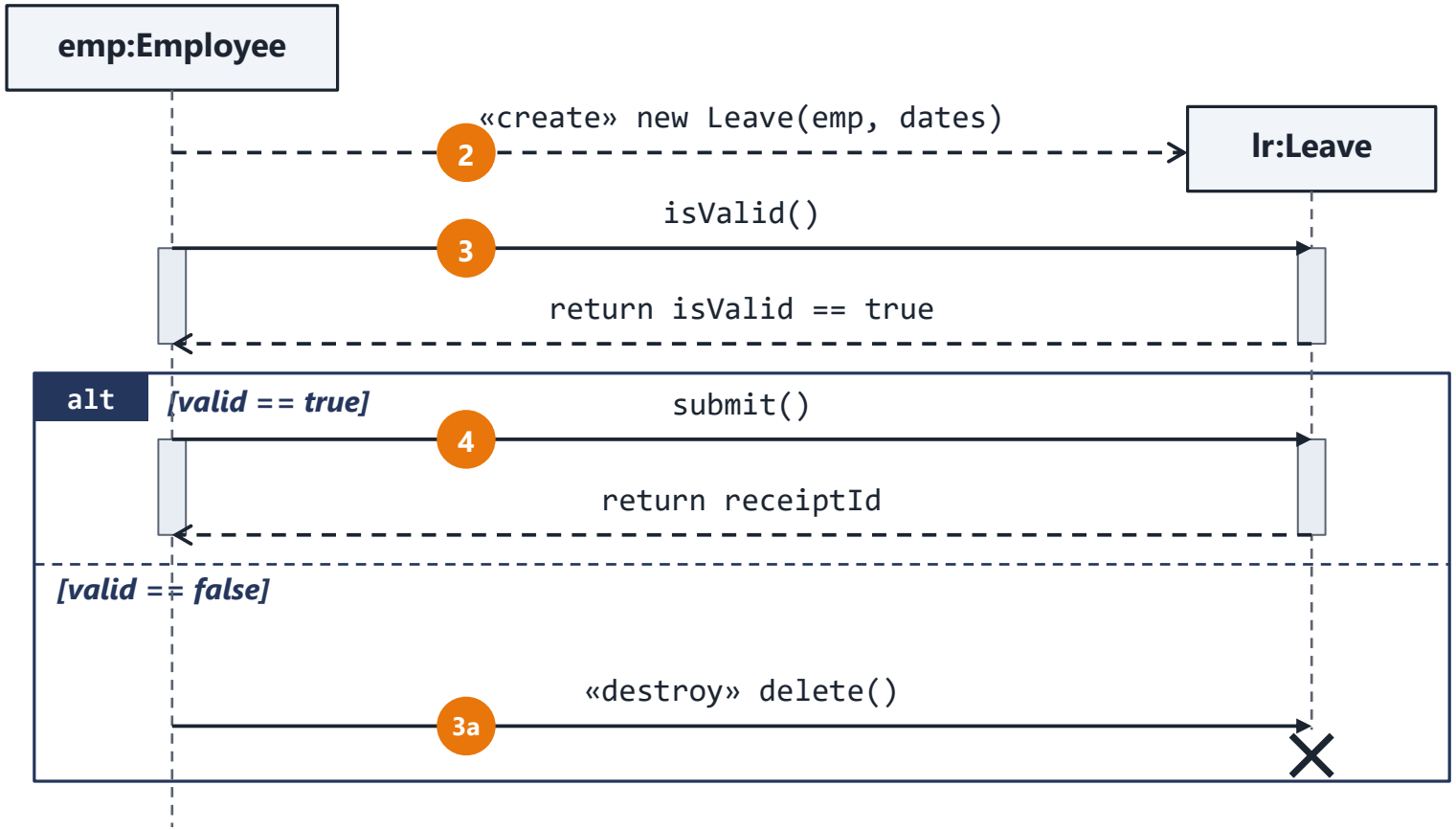
SECTION 3

From use case to working diagram

the LMS centerpiece · reading it · matching it to code

UC-01 Request Leave, end to end

numbered chips = the use-case step each message realizes



Read top → bottom

② create: lr is born
③ sync call + reply: validation
④ / 3a: alt fragment picks ONE region
X : lr's life ends

Code lab: match code to arrows

which statement produces which message? (numbers on the diagram)

EmployeePortal.java · design-level realization of UC-01

```
public LeaveRequest requestLeave(Employee emp, DateRange dates) {
    LeaveRequest lr = new LeaveRequest(emp, dates);   // A
    if (lr.isValid()) {                               // B
        lr.submit();                                   // C
        return lr;
    } else {
        lr.cancel();                                   // D
        throw new InvalidLeaveRequest(dates);
    }
}
```

Messages on the diagram

① `«create» new Leave(emp, dates)`
create

② `isValid()  →  return boolean`
sync + reply

③ `submit()`
sync (alt region 1)

④ `«destroy» / cancel path`
alt region 2

Match A–D to ①–④, then check the if/else ↔ alt correspondence.

Use-case flows → diagram structures

your use case text already tells you which fragments you need

Main flow steps	→	**sequence of messages, top → bottom**
Alternative flow (“if invalid…”, “else…”)	→	**alt / opt fragment**
“for each …”, “repeat until …”	→	**loop fragment**
“…at the same time / concurrently…”	→	**par fragment**
Included sub-use case («include»)	→	**ref (interaction use) to another diagram**

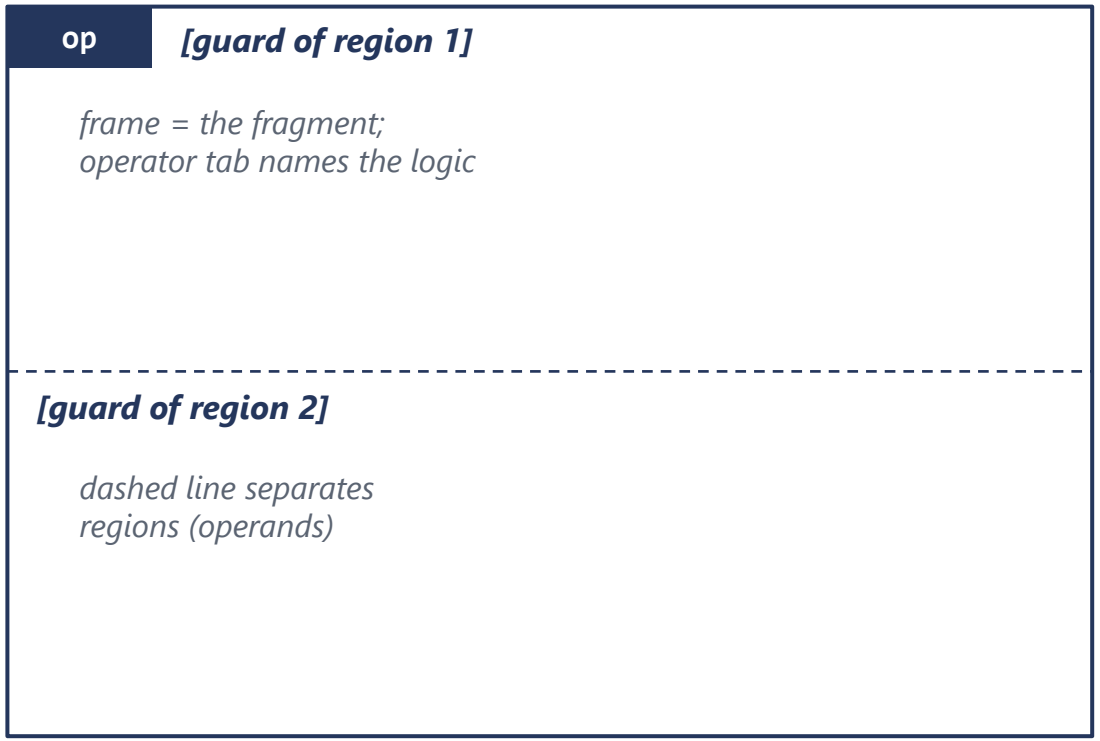
Writing the use case carefully is half the diagram, the other half is choosing the participants.

SECTION 4

Interaction fragments: logic in diagrams

alt · opt · loop · par · ref + activations & state invariants

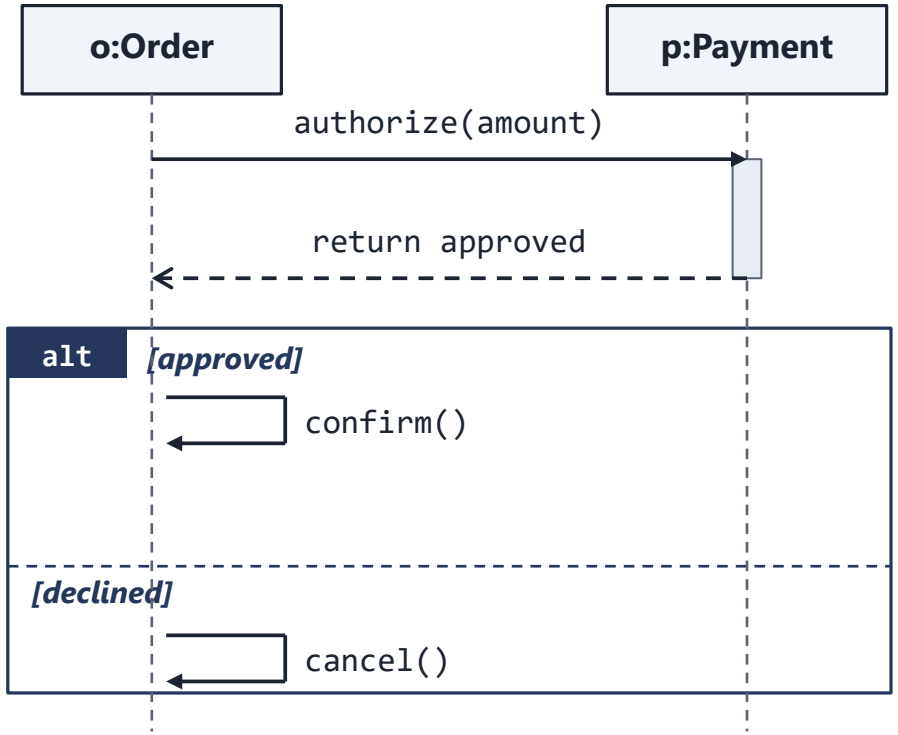
Anatomy of a combined fragment



- A combined fragment = a named frame containing **interaction operands** (regions).
- Operators we use today: `alt`, `opt`, `loop`, `par` and the `ref` interaction use.
- UML 2.5 defines more (break, critical, neg, strict, assert…) — same frame idea, different operator word.
- Guards in square brackets are **mutually exclusive** in alt: exactly one region runs.

Alt: exactly one region runs

if / else if / else



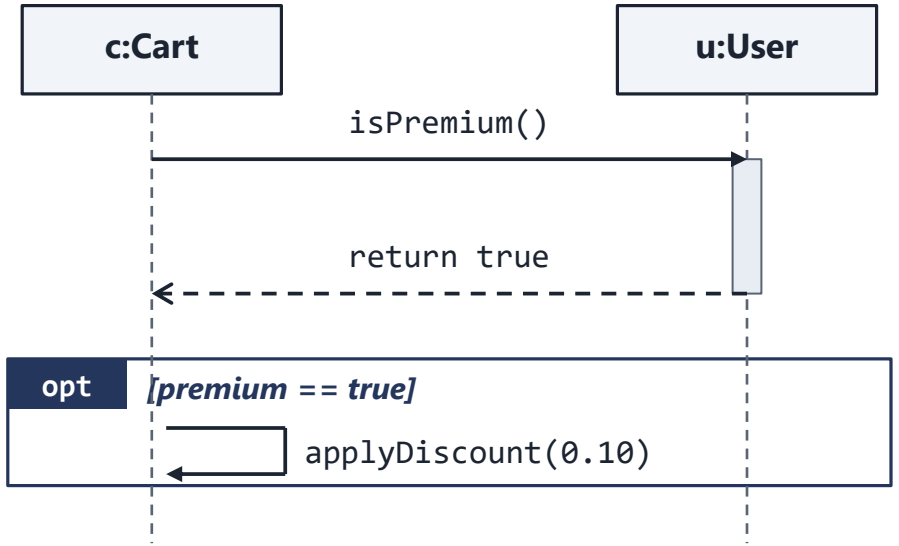
Checkout.java

```java
if (payment.authorize(amount)) {
    order.confirm();
} else {
    order.cancel();
}
```

Two regions, two guards, ONE execution path, the diagram and the if/else say the same thing.

Opt: run the region, or skip it

a single-guard if



Cart.java

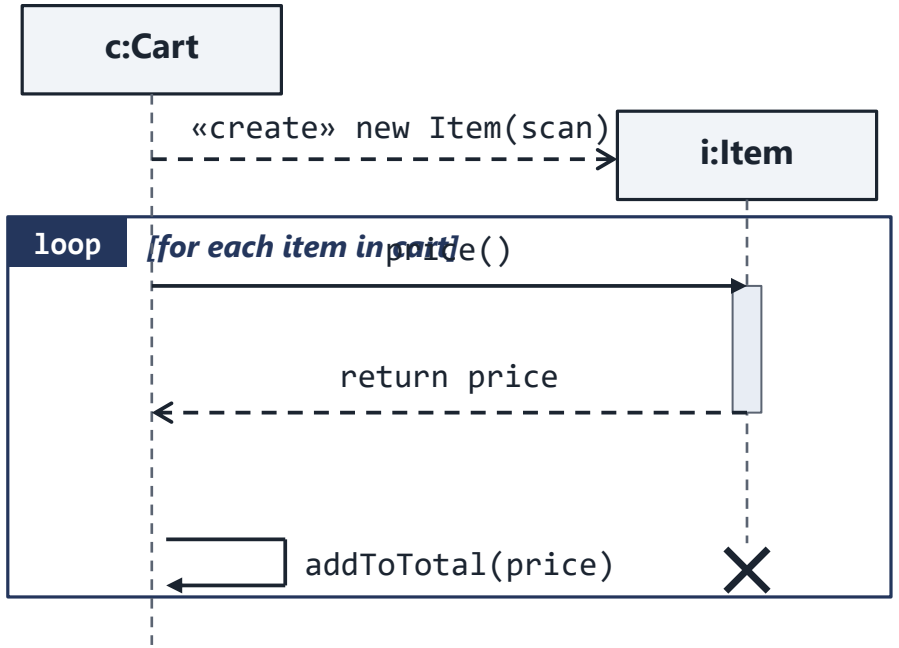
```java
if (user.isPremium()) {
    cart.applyDiscount(0.10);
}
```

opt = alt with one region and an implicit "else: do nothing".

Real situations: loyalty discount, gift wrapping, sending a promo push, anything optional in the flow.

Loop: repeat the region

guard states the iteration condition or bound



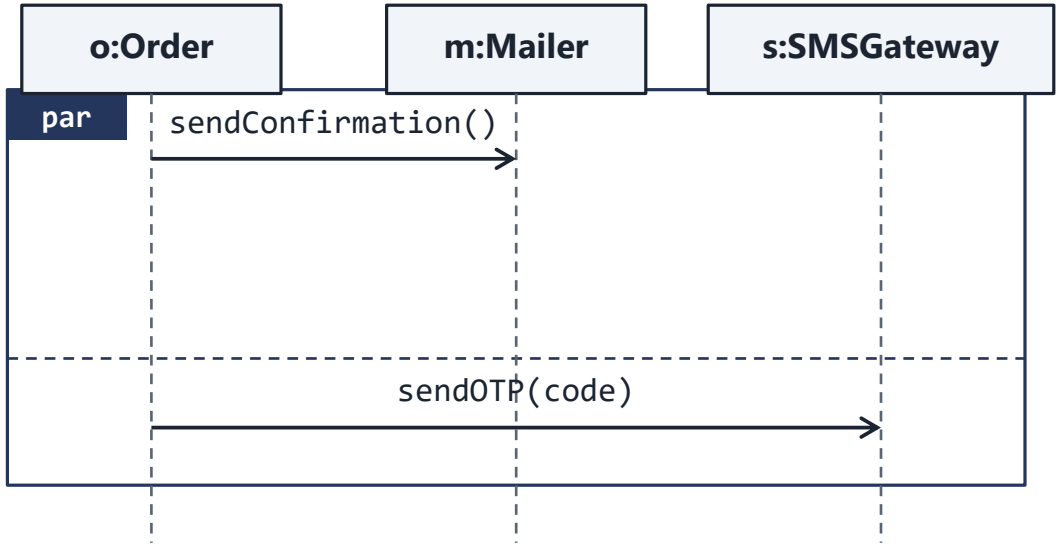
Cart.java

```
for (Item item : cart.items()) {
    total += item.price();
}
```

Bounds can be explicit: loop [3 times] or loop [until validated]. Same frame, guard tells the rule.

Par: regions run in parallel

interleaved in any order, all regions complete



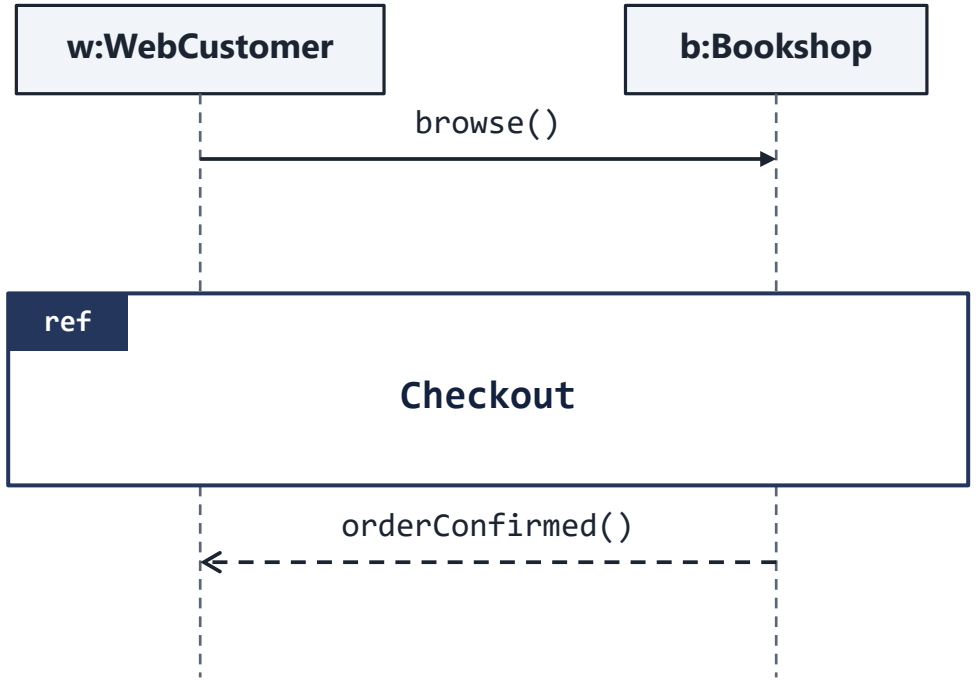
Order.java

```
var email = CompletableFuture.
    .runAsync(() -> mailer.send(o));
var sms = CompletableFuture.
    .runAsync(() -> gateway.sms(o));
CompletableFuture.allOf(email, sms)
    .join();
```

Caution: par means the regions may interleave in ANY order. If order matters, you do NOT have par, you have a plain sequence (or 'strict').

Ref: reuse another interaction

interaction use: call a diagram defined elsewhere

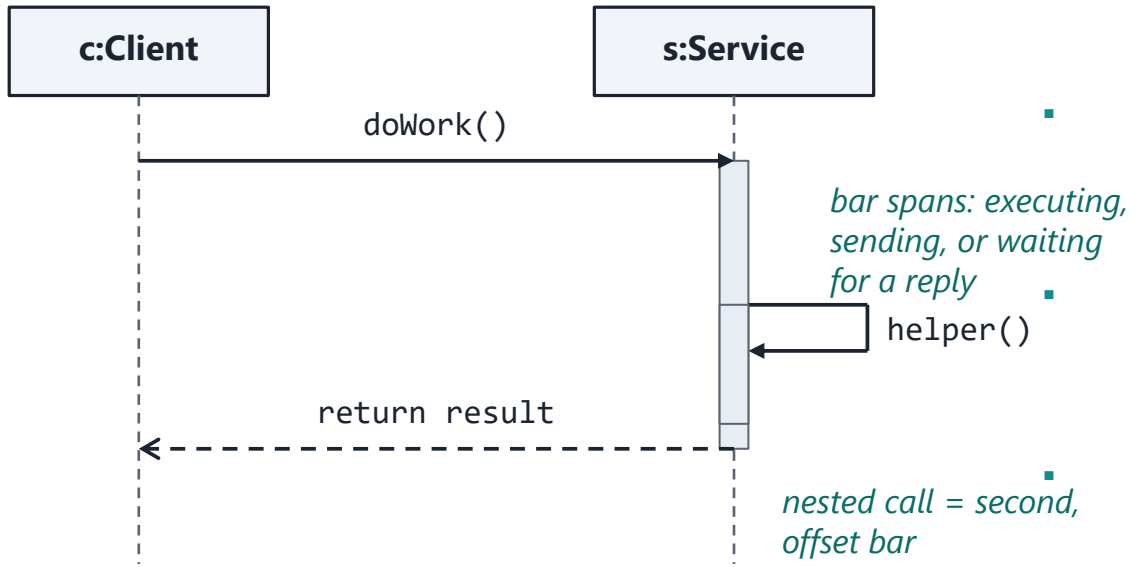


- A **`ref`** frame calls another sequence diagram by name, like a function call between diagrams.
- Keeps large diagrams readable: the checkout conversation lives on its own page.
- Pairs with use cases: an **`«include»`**d sub-use case usually becomes a ref.

Classic example: WebCustomer and Bookshop both participate in the referenced interaction "Checkout", defined once and reused.

Execution specifications (activations)

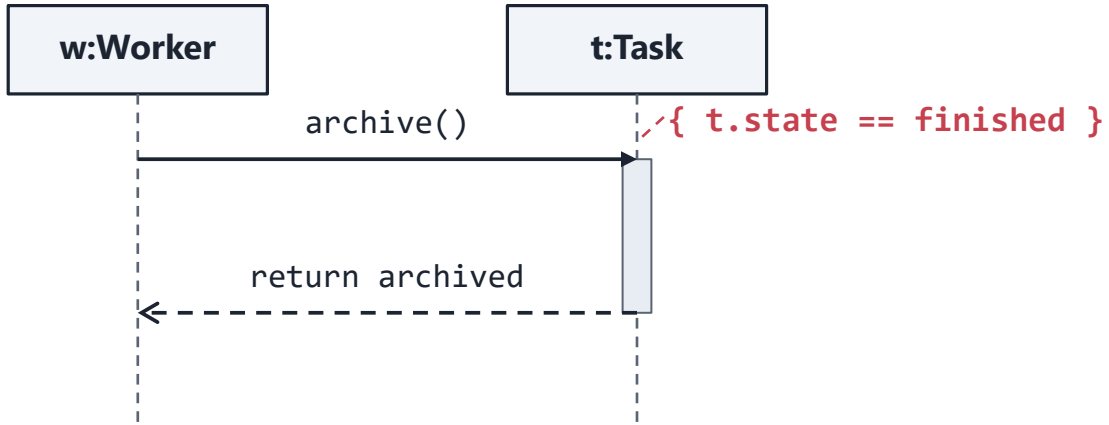
the thin bar = "this object is busy"



- An execution specification marks the period a participant is executing behavior, sending a signal, or waiting for a reply.
- Overlapping bars on one lifeline = nested calls (self-messages, callbacks).
- Style note: many teams omit bars on simple diagrams; include them when control flow is the point.

State invariants: rules on a lifeline

a run-time constraint that must hold at that point



- A state invariant is a run-time constraint on a participant, attribute values, internal state, etc.
- Drawn in curly braces on the lifeline; execution may only reach that point if the constraint holds.
- Example: archiving a Task is only legal when **`t.state == finished`**.

Think of it as an assertion on the timeline:
`assert t.state == 'finished'`

SECTION 5

Sequence diagrams in industry today

diagrams-as-code · where teams actually use them · a microservice case

Diagrams-as-code

modern teams store diagrams as TEXT next to the code

- **Mermaid**: markdown-style text definitions; GitHub renders ```` ```mermaid ```` fenced code blocks directly in READMEs, issues and pull requests.
- **PlantUML**: text → UML renderer (plantuml.com), widely used with wikis and CI pipelines.

- Consequence: diagrams get **versioned, diffed and code-reviewed** like source files, they rot less.

balance.md — rendered by GitHub on push

```
sequenceDiagram
    participant C as c:Customer
    participant B as b:Bank
    C->>B: checkBalance(pin)
    B-->>C: return balance
```

Arrow vocabulary (Mermaid)

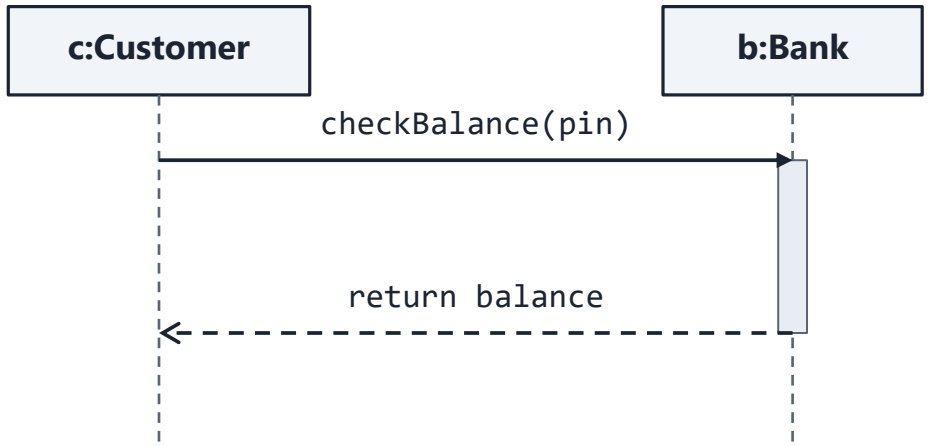
```
->>   solid + filled head
-->>  dashed + head (reply)
-)    solid + open (async)
-x    solid + cross (lost)
```

One model, two notations

left: the text you commit · right: what UML draws

balance.mmd

```
sequenceDiagram
    participant C as c:Customer
    participant B as b:Bank
    C->>B: checkBalance(pin)
    B-->>C: return balance
```



Try it live: mermaid.live: paste, render, screenshot into your docs.

The text is the source of truth; the picture is a build artifact.
That is the docs-as-code idea in one sentence.

Where you'll meet them at work

Microservice & API design

design reviews draw service-to-service call order before code exists, sync vs async choices become visible, reviewable decisions.

Onboarding & runbooks

“how a request flows through our system” is the first doc new engineers read; a sequence diagram is that doc.

Incident postmortems

reconstructing “who called whom, who timed out” is literally re-drawing a sequence diagram of the failing scenario.

Cross-team contracts

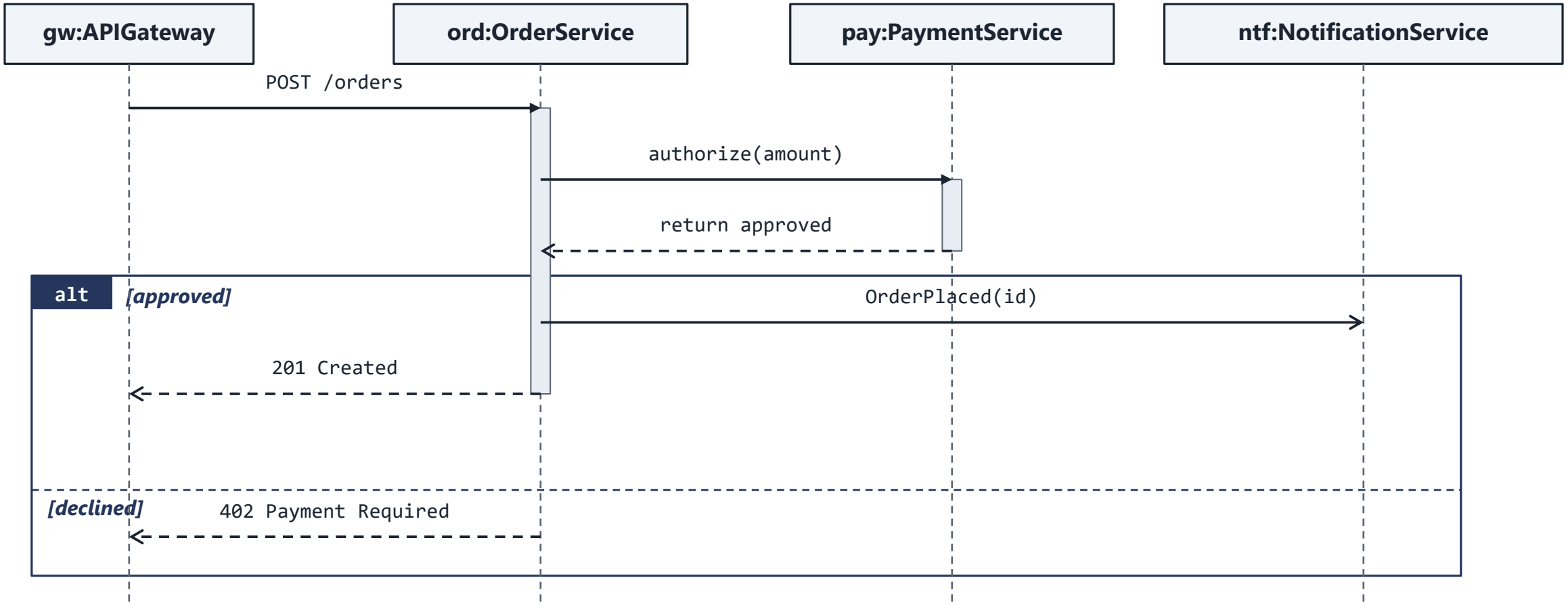
the client–server contract from Section 2, written down: team A's arrow into team B's lifeline is a promise.

Code → diagram tools

IDE plugins (e.g., SequenceDiagram.org for IntelliJ/Android Studio) and UML tooling generate sequence views from existing code to understand it.

Case: placing an order across services

a design-review diagram you might really write



Discuss: which of these arrows would you make asynchronous in a real system, and what would that change for the user experience?

SECTION 7

Wrap-up

takeaways · rookie mistakes · exit ticket · resources

Key takeaways

- Sequence diagrams show how objects collaborate over time, time flows top → bottom.
- Lifelines are participants (one instance each); messages are the communication between them.
- Message types: synchronous, asynchronous, reply, create, delete (+ self). The arrowhead says who waits.
- Fragments add logic: alt / opt / loop / par, plus ref to reuse whole interactions.
- Use cases feed diagrams: main flow → messages, alternative flows → fragments (use-case realization).
- Every arrow has a code twin: a call, a return, a new, an event publish, read diagrams as deferred code.
- Industry keeps them alive as text: Mermaid on GitHub, PlantUML, design docs, postmortems.

Five mistakes (exam & life)

LECTURE

1. **Drawing a flowchart**
   no decision diamonds here, decisions live in alt/opt guards on messages.

2. **One diagram for the whole system**
   scope = ONE scenario of ONE use case; split with ref when it grows.

3. **A lifeline per class**
   lifelines are instances in this conversation; spectators stay off stage.

4. **Forgetting who waits**
   sync vs async is a DESIGN DECISION with real cost, the arrowhead records it.

5. **Unlabeled or decoration arrows**
   every arrow carries a message name (and arguments when they matter).

Resources & tools

`uml-diagrams.org`	the notation reference used to cross-check today's slides (sequence diagrams section)
`OMG UML 2.5.1 specification`	the standard itself, section 17 (Interactions) when you want the exact semantics
`mermaid.live`	type text, get a diagram instantly; GitHub renders the same syntax in markdown
`plantuml.com`	text → UML renderer, scriptable in CI
`diagrams.net (draw.io) · lucidchart.com`	free-form drawing for whiteboarding and reports

Notation in this deck follows UML 2.5; arrow styles cross-checked against uml-diagrams.org.